GUIDE / 01

CommLab Copilot

Build, run, verify. Explain with evidence.

A practical user and engineering guide to the wireless experiment workspace. The main contribution is the software platform: typed tools, durable execution, provenance, reproducibility and bounded investigation.

Readiness at a glance

Ready for a local demonstration and further development. The native SQLite workflow has been exercised end to end. Production deployment, real PostgreSQL/Redis execution and live-model quality evaluation remain incomplete.

The default experience uses explicit offline rules and lexical hashing. It is not a live LLM or a measured semantic-retrieval system.

Inside this guide

Page	Topic
2	Install and start
3	Run the complete demonstration
4	Architecture and data ownership
5	Scientific assumptions and interpretation
6	Retrieval and investigator workflow
7	Reliability, access and troubleshooting
8	Verification and evaluation evidence
9	Extend the platform
10	Roadmap, interview summary and source map

Edition: 21 October 2026. Based on the repository and its recorded verification report. This guide does not certify new execution or independent scientific review.

Install and start

Use Python 3.12 and Node 22 or 24. Commands below are for PowerShell, starting from the project root. The native path uses SQLite and a local outbox worker.

```
cd D:\CommLab-Copilot
py -3.12 -m venv .venv
.\.venv\Scripts\python -m pip install -r backend/requirements.lock
.\.venv\Scripts\python -m pip install --no-deps -e backend
cd frontend
npm ci
cd ..
.\scripts\start-local.ps1
```

If dependencies are already installed, only run the start script. It refuses to start when ports 8000 or 5173 are occupied. Open http://127.0.0.1:5173; interactive API documentation is at http://127.0.0.1:8000/docs.

Stop the recorded background processes with **scripts/stop-local.ps1**. Logs and process records are under backend/data/. The script checks process start times to avoid stopping a reused process ID.

Alternative: Docker Compose

```
Copy-Item .env.example .env
docker compose up --build
```

Open http://localhost:8080. Compose defines PostgreSQL/pgvector, Redis, migrations, API, dispatcher, dedicated workers and frontend. Configuration was validated; containers were not executed because the development machine lacked a running Docker engine.

Configuration and credentials

Native processes read shell environment variables; they do not automatically load .env. Leave DATABASE_URL unset for SQLite. API and worker must share the same database and artifact root. Compose reads the root .env file.

Create a workspace in the browser and export its credential to reconnect later. This JSON contains a bearer secret; keep it private. For seeded data, run scripts/seed_demo.py from backend using the project Python, then import backend/data/demo-credential.json through the reconnect control.

Run the complete demonstration

1. Add evidence

Create a workspace, upload samples/synthetic-bpsk-lab.md, and wait for ready. This handout is clearly labelled synthetic. PDF, TXT and Markdown are supported within a 2 MB file limit.

2. Ask a supported question

Ask: "Compare BPSK over AWGN and Rayleigh using this handout. Check consistency with theory." Open a returned citation to inspect its exact source version and text.

3. Review before execution

Inspect the two proposed configurations. Review channel, Eb/N0 points in dB, bits per point and seed. Offline mode uses narrow patterns and documented defaults; arbitrary parameter instructions are not guaranteed to be interpreted.

4. Approve and observe

Approve the plan. Jobs move through queued and running to a terminal status. Progress and cancellation are available. The investigator yields while simulations run.

5. Compare and explain

Select the two completed runs. Inspect measured and theoretical curves, error counts, confidence intervals, changed configuration fields and verification findings. Download JSON and SVG artifacts.

6. Reproduce

Choose Reproduce to load a saved configuration, then submit a new run. Check simulator and runtime versions before claiming exact repeatability. Equal seeds alone are insufficient.

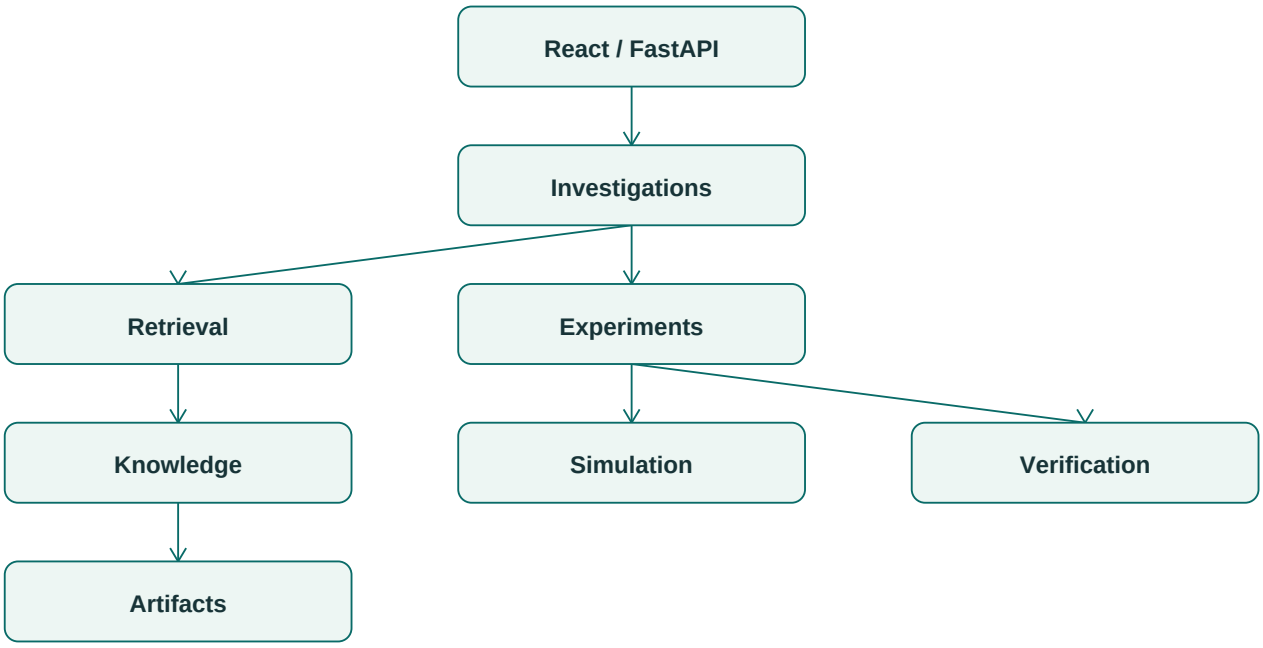
What the recorded demo actually observed

At 0 dB, seed 42	AWGN	Rayleigh
Bits simulated	100,000	100,000
Measured BER	0.08058	0.14530
Theoretical BER	0.0786496	0.146447

These are recorded Monte Carlo outcomes, not guaranteed future values. At 8 dB the demonstrated AWGN run counted 27 errors, triggering a precision caution. The original browser capture is saved in docs/browser-comparison.png.

Architecture and ownership

A modular monolith keeps ordinary interactions as direct calls. Separate workers handle parsing, simulation and model calls because their execution needs differ. Application assembly injects replaceable providers.



The diagram shows the main call path. Access checks guard HTTP routes. Investigations and Experiments also use Artifacts; Verification consumes Simulation contracts. Knowledge, Experiments and Investigations enqueue durable outbox work. Workers call their public APIs.

Module	Owns
Workspaces	Workspace identity and owner credential hash; membership is future work.
Knowledge	Immutable source versions, chunks, parsing status and embedding identity.
Retrieval	Filtered search requests, ranking and evidence results.
Experiments	Run lifecycle, idempotency, leases, configurations and saved result snapshots.
Simulation / Verification	Numerical contracts and backend / deterministic finding rules.
Investigations	Plans, approvals, tool history, budgets, usage and answers.
Artifacts / Jobs	Blob metadata and checksums / durable outbox delivery records.

Each module exposes api.py. Other modules must not import its ORM classes or access its tables. The import-boundary test protects declared restrictions; review remains necessary. docs/architecture.md records intentional exceptions, including shared transaction access for the outbox.

Scientific assumptions

The implemented simulator is narrow NumPy Monte Carlo code checked against independently expressed analytic formulas. It is not an externally certified simulator. Uploaded text cannot change its scientific conventions.

Convention	Implemented meaning
Energy and SNR	BPSK has one bit per symbol: Eb = Es = 1. gamma = 10^(Eb/N0_dB / 10); N0 = 1/gamma.
AWGN	y = x + n; x is -1 or +1. Real Gaussian noise variance is N0/2.
Rayleigh	y = h*x + n; h = (a + j*b)/sqrt(2), independent standard normals a,b per symbol. E|h|^2 = 1.
Receiver	Perfect channel knowledge; decision uses Re(conj(h)*y). Complex noise has variance N0/2 per real component.
Excluded effects	No coding, channel-estimation error, Doppler correlation, block fading, interference or diversity.

Independent theoretical references

```
AWGN BER     = 0.5 * erfc(sqrt(gamma))
Rayleigh BER = 0.5 * (1 - sqrt(gamma / (1 + gamma)))
```

Interpret uncertainty correctly

Each point uses a fixed bit count. Results store errors, bits, BER and exact two-sided 95% Clopper-Pearson intervals. Zero errors gives a zero point estimate with a positive upper bound; logarithmic plots show that bound separately.

Theory verification uses a 1% familywise significance level with Bonferroni correction across points. A discrepancy is evidence of inconsistency, not proof of a particular bug. Fewer than 30 errors triggers a separate sample-precision warning. Curves need not be perfectly monotonic.

Reproducibility envelope and limits

Results record schema and implementation versions, seed, Python/NumPy/SciPy, platform, PCG64 and batch size 65,536. Exact count reproducibility is supported within the same implementation/runtime envelope. Timing and SVG metadata need not match.

Limits: -10 to 25 dB; 1-21 increasing unique points; 1,000-2,000,000 bits per point; 10 million bits total; cooperative 120-second simulation deadline.

Evidence and investigation

Two retrieval paths

Reference retrieval uses keyword and vector candidates with reciprocal-rank fusion, constrained by workspace, source version and compatible embedding/index identity. The offline embedding is lexical hashing, not a learned semantic model. PostgreSQL-specific search is supplied but remains unexecuted locally.

Experiment retrieval uses exact structured fields such as seed and channel. It does not substitute vector similarity for exact database filters.

Citations resolve to immutable stored sources, chunks and checksums. Page and section are preserved where available. PDF equation extraction is flagged as unvalidated; OCR and automatic formula execution are absent. Insufficient evidence remains visible.

Persisted workflow

```
question -> retrieve / inspect -> typed plan -> user review
         -> approve -> submit supported jobs -> wait
         -> inspect results -> deterministic checks -> answer
```

One bounded decision executes at a time. State, tool activity, plans, run IDs and budgets persist. No model request stays open while a simulation runs. Answers distinguish source quotations, observed results, hypotheses and unresolved questions; the timeline exposes activity rather than hidden reasoning.

Budgets: 12 tool/history steps, 12,000 reported model tokens, 180 seconds cumulative active work, 30 minutes active wall time, at most two proposed simulations and two transient provider retries. Human review time is excluded. Token accounting is checked between calls.

Optional real providers

```
OLLAMA_URL=http://localhost:11434
LLM_MODEL=<installed-chat-model>
EMBEDDING_MODEL=<installed-384-dimensional-model>
EMBEDDING_REVISION=<exact-model-digest>
```

Use an existing suitable server; no model is downloaded automatically. LLM and embedding adapters can be enabled independently. The embedding adapter verifies the digest and dimensions. After changing identity, reingest into new source versions. No live-model quality or cost measurements are claimed.

Reliability and operations

Run lifecycle

```
queued -> running -> succeeded | failed | cancelled
queued -> cancelled
running -> queued  (retryable failure or expired lease)
```

Submission and its outbox record commit atomically. Delivery is at least once. Atomic claims, idempotency keys and lease-fenced publication prevent duplicate or stale publication. Terminal runs are immutable; reproduction creates a new run.

The dispatcher repairs queued work after 60 seconds and expired running leases after 180 seconds. Three execution attempts are allowed for retryable failures. Cancellation is checked between simulation batches. A crash can leave an unreferenced artifact, but a partially published result is not exposed as success. Automated orphan cleanup is future work.

Access and observability

Workspace bearer authorization protects sources, retrieval, runs, investigations and artifacts. The database stores token hashes. This is a single-owner MVP, not team RBAC. Workers execute registered capabilities only; retrieved instructions never authorize code execution.

JSON logs carry correlation IDs and omit private document bodies and bearer tokens by default. Tool records store scoped evidence. /metrics requires METRICS_TOKEN. Worker metrics aggregation, alerting, backups, rate limits and stronger parser supervision remain deployment tasks.

Troubleshooting

Symptom	Action
UI loads; API fails	Check port 8000 and backend/data/api-error.log. Start API and worker with the same database configuration.
Queued work stays queued	Check worker logs and that the worker is running. For Compose, also inspect dispatcher, Redis and migrations.
Upload fails or has no text	Check type/size limits. Scanned or encrypted PDFs are unsupported; use a text-bearing PDF or text export.
No useful evidence	Confirm source is ready, the right workspace/version is selected and embedding identity matches. Do not force a cited answer.
Embedding revision error	Restore the configured digest or intentionally switch identity and reingest sources. Do not mix incompatible vectors.
Port already occupied	Stop the previously recorded local processes, or use manual startup with explicitly coordinated ports.

What has been verified

The following is the recorded repository verification snapshot from 21 October 2026. Generating this guide did not rerun the application test suite.

Check	Recorded outcome
Backend tests	28 passed, 1 skipped; 7.11 seconds. The skipped test requires real PostgreSQL/Redis.
Frontend	TypeScript and production build passed; generated API types refreshed.
Dependency audits	npm and Python lockfile audits reported no known vulnerabilities at that time.
Database / deployment	SQLite up/down/up migration passed. Compose syntax and PostgreSQL SQL compilation passed; real containers were not run.
Evaluation	4/4 calibration and 12/12 held-out deterministic cases passed.
Product walkthrough	Upload, ingestion, citation resolution, review, simulation, comparison and finished investigation exercised in the browser.
Live AI benchmark	Not run. No configured model server; basic LLM, RAG-only and full-agent scores are not available.

Repeat the checks

From backend/, using the project Python:

```
..\.venv\Scripts\python -m pytest -q
..\.venv\Scripts\python scripts/evaluate.py --split calibration
..\.venv\Scripts\python scripts/evaluate.py --split heldout
..\.venv\Scripts\python scripts/export_openapi.py
```

From frontend/: run npm run types, npm run build and npm audit. For the real queue smoke test, migrate an isolated PostgreSQL database, configure DATABASE_URL and REDIS_URL, set COMMLAB_TEST_POSTGRES=1 and run tests/test_postgres_queue.py.

Limits of the evidence

Held-out reference hit@3 was 1.0 on only three synthetic questions. Exact citation text support is not semantic entailment. Two scientific cases reproduced counts without theory alarms. The injected normalization fault was detected as inconsistent, not uniquely diagnosed.

The corpus is tiny and author-reviewed, not independently reviewed. Diagnosis accuracy and model cost remain null. Run scripts/evaluate_models.py with a real provider and use eval/rubric.md for human scoring. Do not present deterministic offline results as an LLM benchmark.

Extend without broad rewrites

Public protocols exist at genuine replacement boundaries: simulator, verification rule, LLM, embedding provider, retrieval strategy and blob storage. assembly.py selects implementations. Stable typed contracts keep unrelated modules isolated.

Change	Expected edits and validation
QPSK	Simulation/spec, applicable theory and forms. Test mapping and Eb/Es conventions; version contracts where needed.
New channel	Channel primitive, assumptions, registry and form. Test normalization, receiver knowledge and reference behavior.
Verification rule	Rule plus assembly registration and positive/negative tests. Existing findings UI and simulator remain unchanged.
Replace LLM	Provider adapter and assembly/environment; test structured tools, usage, failures and live quality.
Object storage	BlobStore adapter and assembly; migrate blobs operationally and test checksums, isolation and interrupted upload.
Retrieval strategy	Ranking strategy and assembly; evaluate relevance, abstention and isolation. New embedding dimensions require migration/reindexing.
DSP adapter	Domain spec/result unions, capability, checks and UI. Add read-model fields only where needed; reuse jobs, access and artifacts.
Visualization	Frontend chart and optional export renderer. Reuse result contract if adequate; test axes, units and uncertainty.

A small extension already exercised

SampleAdequacyRule implements the public verification interface. It warns when a point has fewer than 30 observed errors. It is registered next to TheoryRule; the simulator and normal findings UI are unchanged. The extension test uses a real zero-error simulation with a positive uncertainty bound.

Compatibility policy

Specifications and results currently use schema version 1. Keep old readers and immutable artifacts when introducing v2; dispatch explicitly by version. Scientific behavior changes also require an implementation-version change. Future versions are a documented policy, not implemented speculative readers.

For detailed migration impacts, required tests and modules that should remain untouched, see docs/extensions.md. A new domain legitimately needs several contract and UI changes; the registry does not promise zero integration work.

Roadmap and project narrative

Prioritize evidence before breadth

First execute the PostgreSQL/Redis deployment, including process-kill and queue-outage recovery. Then evaluate pinned real chat and embedding providers with independent reviewers. Expand authentic source material with permission and add stronger operational controls before network deployment.

QPSK and receive diversity fit the scientific backend and verification boundaries. OFDM needs a simulator adapter plus FFT/CP, channel and result contracts. Team workspaces need identities, memberships, roles and migrations. Stale-result detection needs explicit source-to-plan dependencies. Budget-aware selection needs a calibrated information/cost policy. Service extraction should follow measured operational need.

A concise interview explanation

"I built an experiment workspace that turns source evidence into a reviewed, typed plan and executes only supported numerical tools. It saves provenance, configuration, runtime and uncertainty so results can be inspected and reproduced. The main contribution is the reliable bridge between language, evidence and computation."

"I chose a modular monolith with separate workers. A transactional outbox, atomic claims and execution leases tolerate duplicate delivery without claiming exactly-once execution. Scientific verification is deterministic and independent of the model. I distinguish verified local behavior from infrastructure and model quality that still need measurement."

Repository source map

Read next	Purpose
README.md	Setup, demo, provider configuration and verification commands.
docs/architecture.md; contracts.md	Dependency graph, entity ownership, public APIs and version policy.
docs/science.md; operations.md	Scientific conventions, uncertainty, recovery and access limits.
docs/extensions.md; roadmap.md	Change-impact matrix, capability guide and future work.
docs/verification.md	Executed checks, browser evidence and remaining gaps.
eval/rubric.md; results-*.json	Evaluation protocol and actual deterministic outputs.
docs/openapi.json	Generated authoritative HTTP contract.
backend/commlab; frontend/src	Working application code; module APIs and separated UI features.

All paths are relative to D:/CommLab-Copilot. This PDF is generated by scripts/build_pdf_guide.py. Runtime secrets and exported workspace credentials are intentionally excluded.